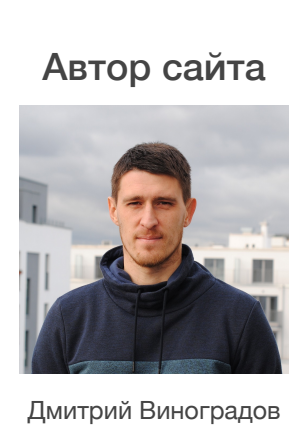
Language

Автор сайта



Дмитрий Виноградов

Подробнее можно посмотреть или почитать.

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.

Курс
Jetpack Compose

Социальные сети

В канале я публикую ссылки на интересные и полезные статьи по Android

В чате можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте

StartAndroid

15 086 подписчиков

Подписаться

Поддержка проекта

Яндекс
410011180491924

WebMoney
R248743991365
Z551306702056

Paypal

Урок 4. Базовые Layout: Row, Column, Box

14 декабря 2022

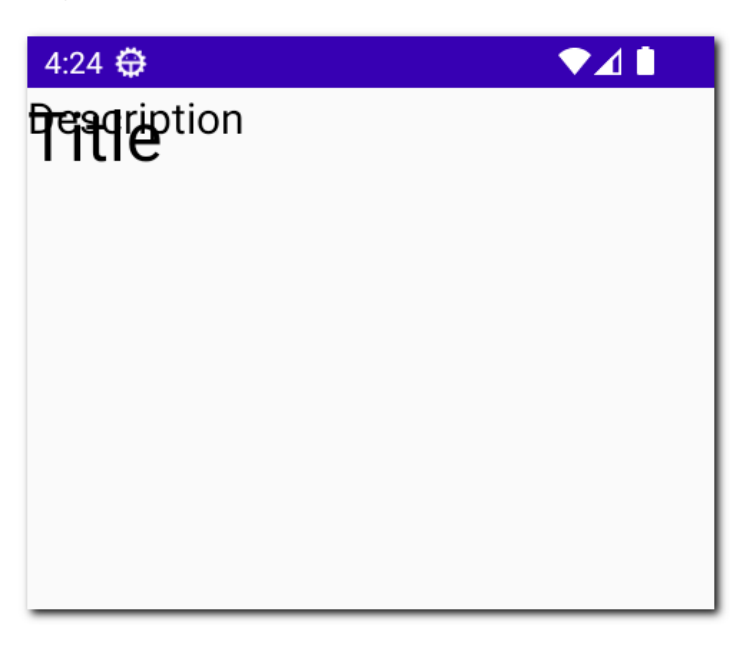
В этом уроке используем базовые Layout для расположения элементов на экране.

В одном из прошлых уроков мы уже пытались показать два текста на нашем Composable экране. Если мы делаем это без Layout, то тексты просто накладываются друг на друга

Пример:

```
@Composable
fun HomeScreen() {
   Text(text = "Title", fontSize = 32.sp)
   Text(text = "Description", fontSize = 20.sp)
}
```

Результат



Title Description

Как и в случае с классическим XML экраном, в Compose нам тоже необходимо использовать Layout, чтобы располагать элементы на экране.

Рассмотрим базовые Layout

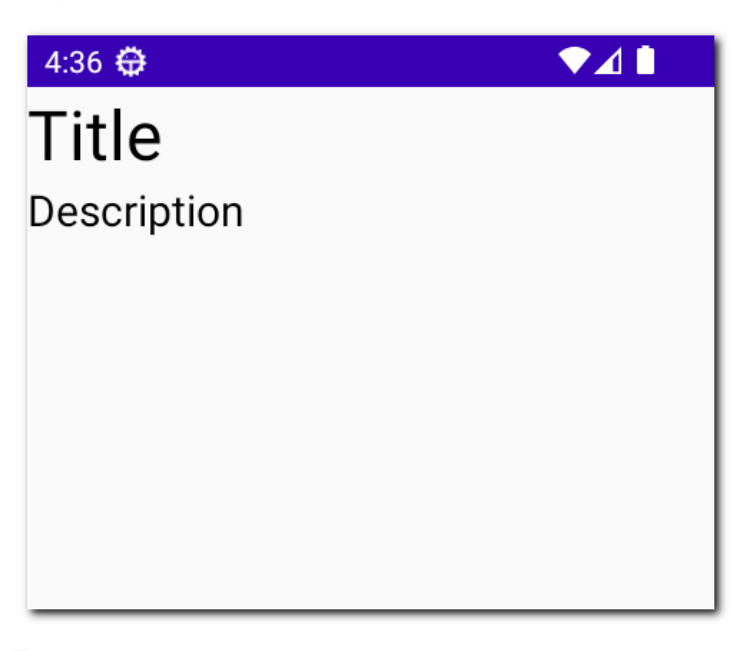
Column

Column - это аналог вертикального LinearLayout. Он выстроит элементы в вертикальный ряд.

```
import androidx.compose.foundation.layout.Column

@Composable
fun HomeScreen() {
    Column {
        Text(text = "Title", fontSize = 32.sp)
        Text(text = "Description", fontSize = 20.sp)
    }
}
```

Результат:



Title
Description

Теперь тексты располагаются один над другим

Выравнивание

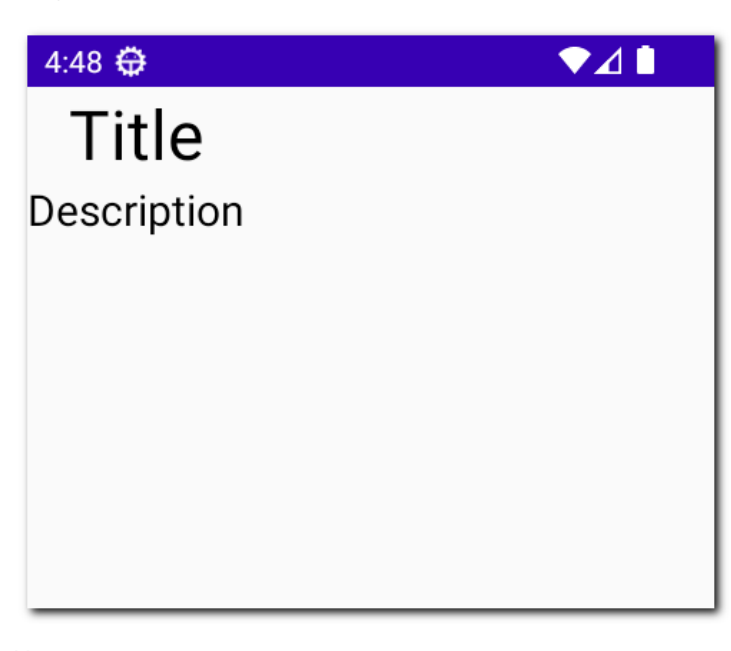
У Column есть атрибуты для выравнивания: вертикального и горизонтального.

Давайте настроим их так, чтобы тексты были внизу экрана и центрированы по горизонтали

```
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.ui.Alignment

@Composable
fun HomeScreen() {
    Column(
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Bottom
    ) {
        Text(text = "Title", fontSize = 32.sp)
        Text(text = "Description", fontSize = 20.sp)
    }
}
```

Результат:



Title
Description

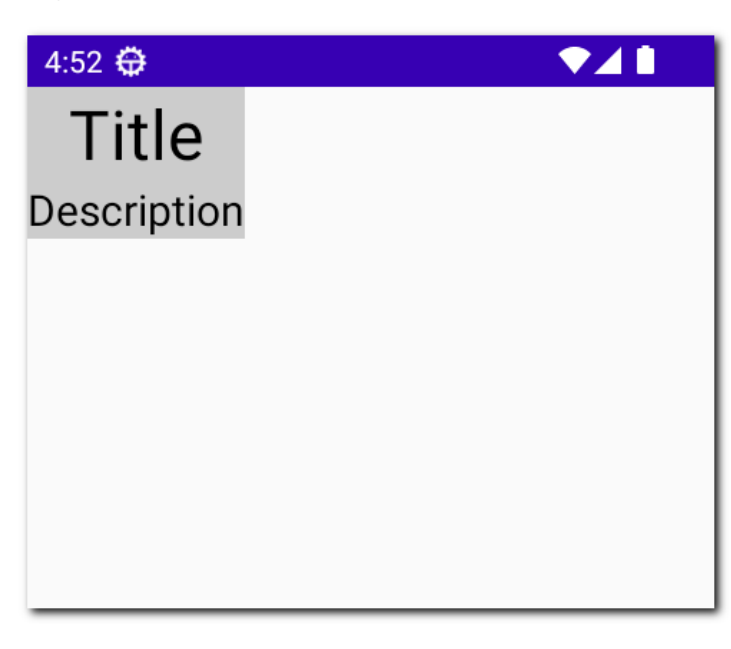
Немного не то, что мы ожидали.

Так получилось потому, что Column по умолчанию не занимает весь экран. Давайте добавим ему фоновый цвет, чтобы убедиться в этом:

```
import androidx.compose.ui.graphics.Color
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.ui.Alignment

@Composable
fun HomeScreen() {
    Column(
        modifier = Modifier.background(color = Color.LightGray),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Bottom
    ) {
        Text(text = "Title", fontSize = 32.sp)
        Text(text = "Description", fontSize = 20.sp)
    }
}
```

Результат:



Title
Description

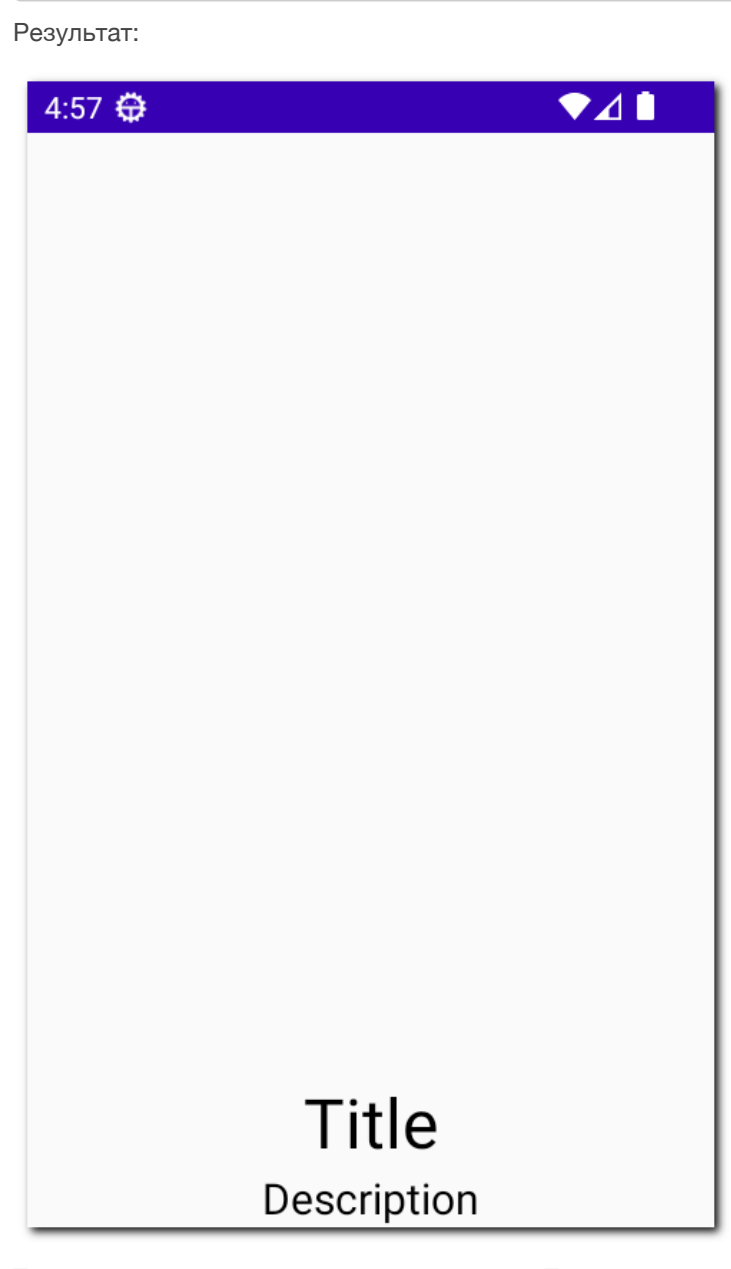
Видно, что Column занимает места ровно столько, чтобы вместить два текста. Т.е. по умолчанию работает в режиме wrap_content. И внутри себя он применил то выравнивание, которое мы попросили.

Надо сделать его на весь экран. Для этого есть Modifier.fillMaxSize:

```
import androidx.compose.foundation.layout.fillMaxSize

@Composable
fun HomeScreen() {
    Column(
        modifier = Modifier.fillMaxSize(),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Bottom
    ) {
        Text(text = "Title", fontSize = 32.sp)
        Text(text = "Description", fontSize = 20.sp)
    }
}
```

Результат:



Title
Description

Теперь получилось то, что мы планировали. Тексты располагаются внизу и выровнены по центру по горизонтали.

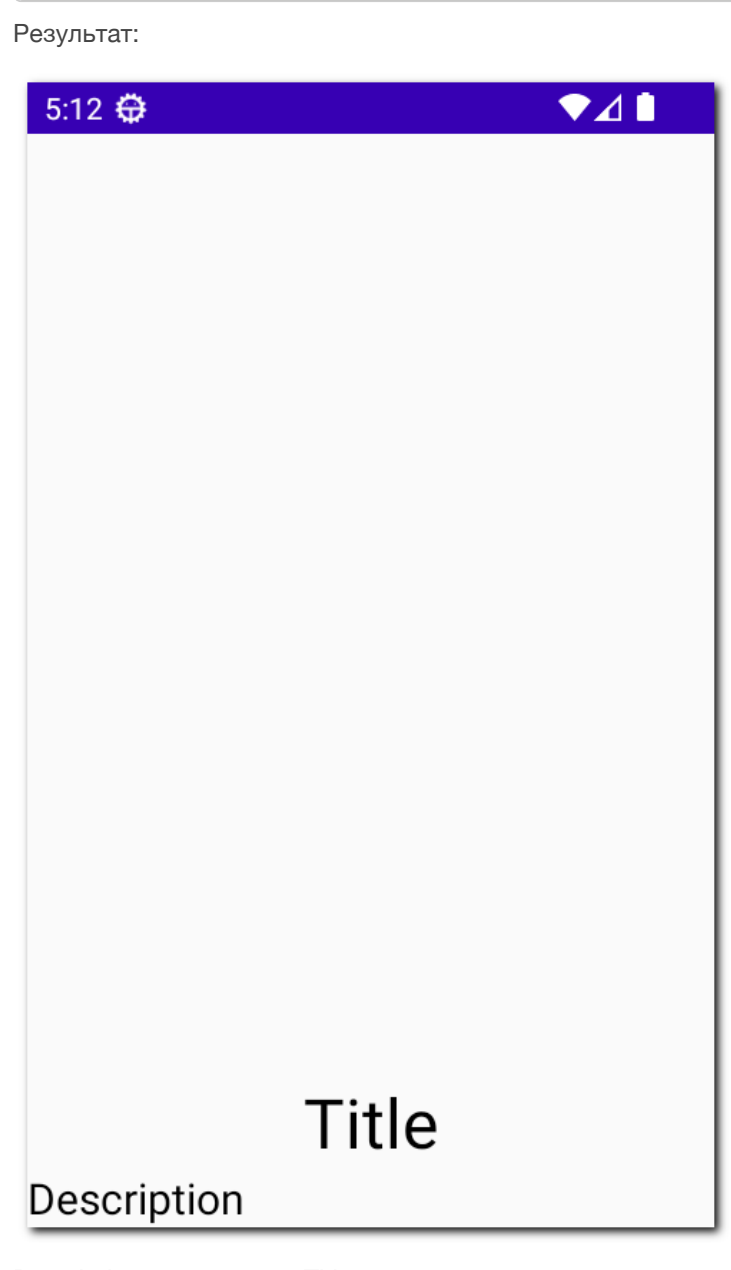
Если нам надо поменять выравнивание для одного из элементов в Column, то мы можем это сделать через Modifier этого элемента. Это будет иметь приоритет перед общим выравниванием в Column.

В качестве примера включим выравнивание по левому краю для элемента с текстом Description:

```
import androidx.compose.ui.Alignment.Companion.Start

@Composable
fun HomeScreen() {
    Column(
        modifier = Modifier.fillMaxSize(),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Bottom
    ) {
        Text(text = "Title", fontSize = 32.sp)
        Text(text = "Description", fontSize = 20.sp,
            modifier = Modifier.align(Start)
        )
    }
}
```

Результат:



Title
Description

Description уехал влево, Title остался по центру

Row

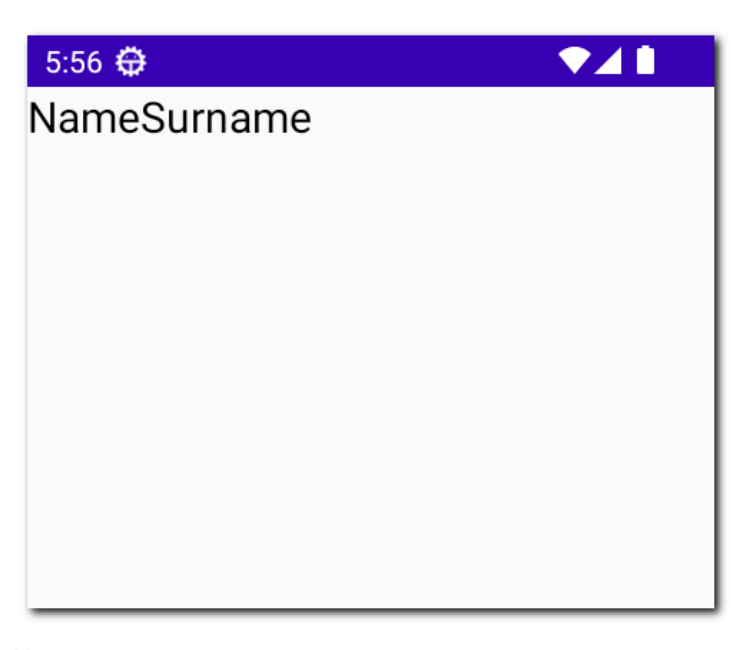
Row - это горизонтальный LinearLayout. Соответственно он полностью аналогичен Column, только выстраивает элементы не по вертикали, а по горизонтали

Поместим два текстовых элемента в Row:

```
import androidx.compose.foundation.layout.Row
import androidx.compose.ui.Alignment.Companion.CenterVertically

@Composable
fun HomeScreen() {
    Row(verticalAlignment = CenterVertically) {
        Text(text = "Name", fontSize = 20.sp)
        Text(text = "Surname", fontSize = 20.sp)
    }
}
```

Результат:



NameSurname

Когда мы располагали тексты по вертикали, между ними визуально было свободное место, потому что у шрифтов есть свои отступы сверху и снизу. А при расположении элементов по горизонтали сразу становится видно, что элементы располагаются вплотную друг к другу. Ни Row, ни Column не вставляют никакие отступы между элементами.

Spacer

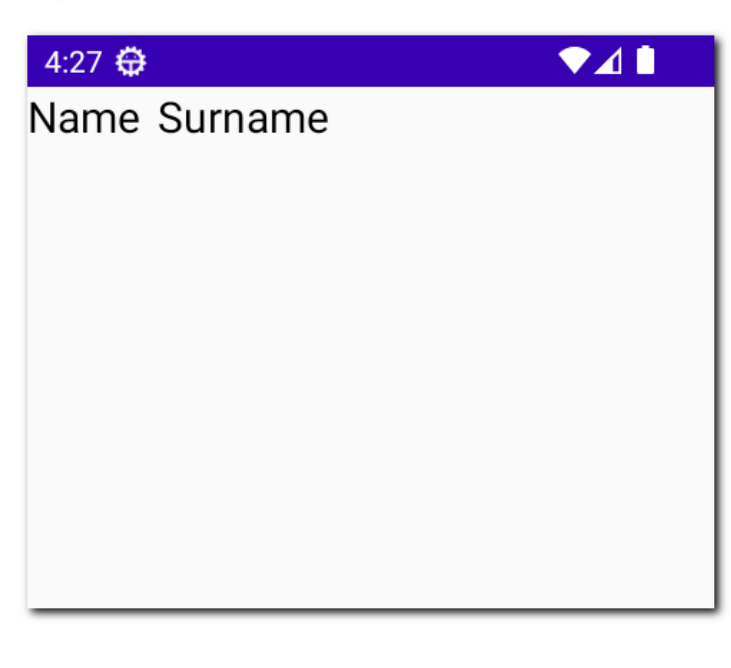
Добавим отступы между элементами используя элемент Spacer:

```
import androidx.compose.ui.Modifier
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.width
import androidx.compose.ui.unit.dp

@Composable
fun HomeScreen() {
    Row(verticalAlignment = CenterVertically) {
        Text(text = "Name", fontSize = 20.sp)
        Spacer(modifier = Modifier.width(8.dp))
        Text(text = "Surname", fontSize = 20.sp)
    }
}
```

Добавили 8 dp пустого пространства между текстами

Результат:

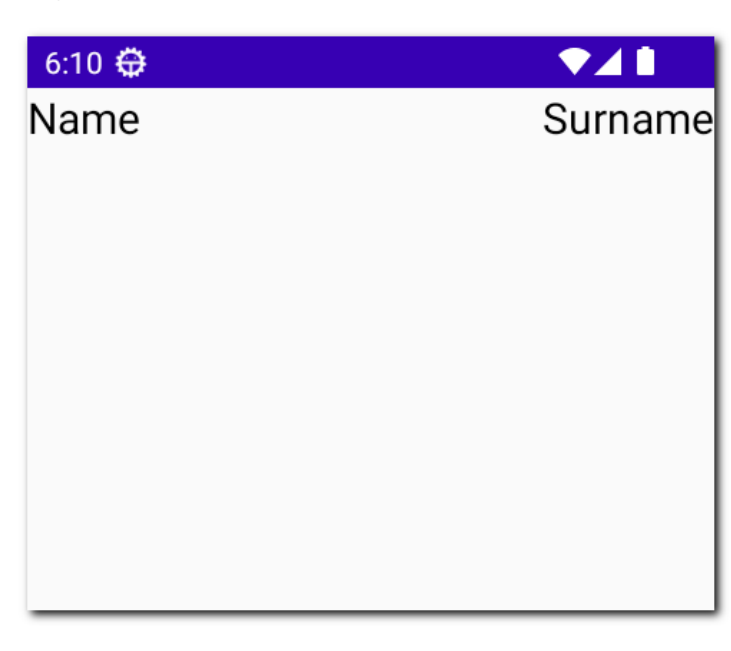


Name Surname

Если нам надо не просто добавить фиксированный отступ, а максимально раскидать элементы, то используем weight:

```
@Composable
fun HomeScreen() {
   Row(verticalAlignment = CenterVertically) {
       Text(text = "Name", fontSize = 20.sp)
       Spacer(modifier = Modifier.weight(1f))
       Text(text = "Surname", fontSize = 20.sp)
   }
}
```

Результат:



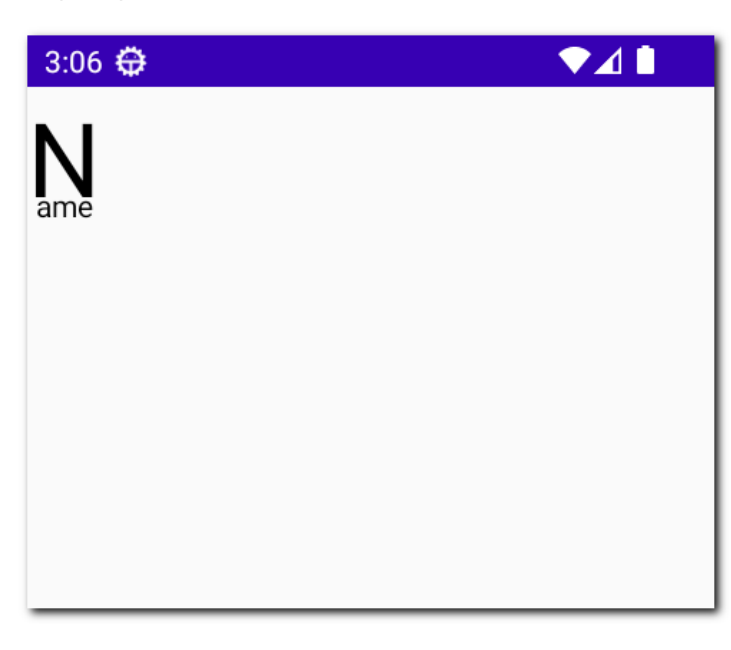
Name Surname

Spacer можно использовать не только в Row, но и в Column

Box

Box - это аналог FrameLayout. Т.е. Layout в котором мы можем накладывать элементы друг на друга.

Например, мы хотим сделать такой текст:

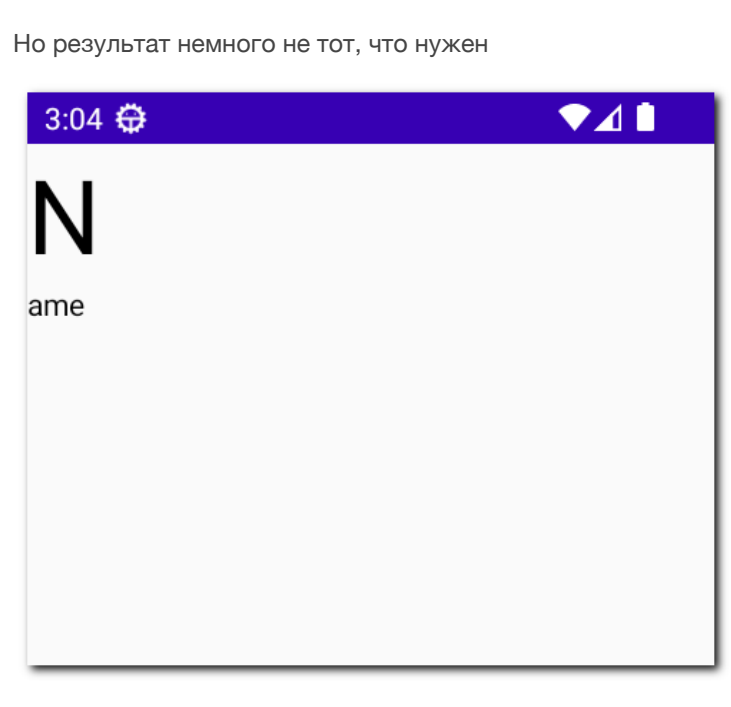


N
ame

Кажется, что можно просто использовать Column:

```
@Composable
fun HomeScreen() {
    Column {
        Text(text = "N", fontSize = 48.sp)
        Text(text = "ame")
    }
}
```

Но результат немного не тот, что нужен

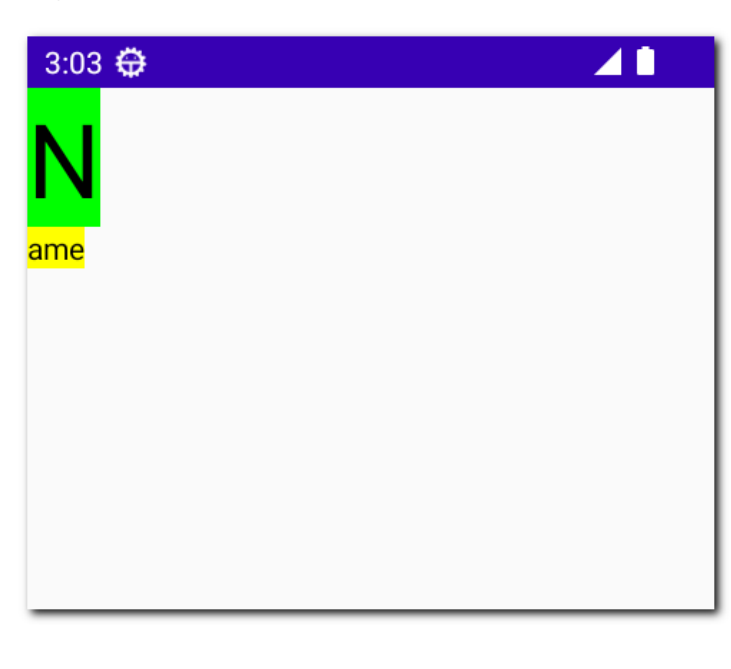


N
ame

Отступы шрифтов не дают нам приблизить два текста вплотную. Чтобы явно увидеть это, можно добавить background к обоим текстам:

```
@Composable
fun HomeScreen() {
   Column {
       Text(text = "N", fontSize = 48.sp,
           modifier = Modifier.background(color = Color.Green)
       )
       Text(text = "ame",
           modifier = Modifier.background(color = Color.Yellow)
       )
   }
}
```

Результат:



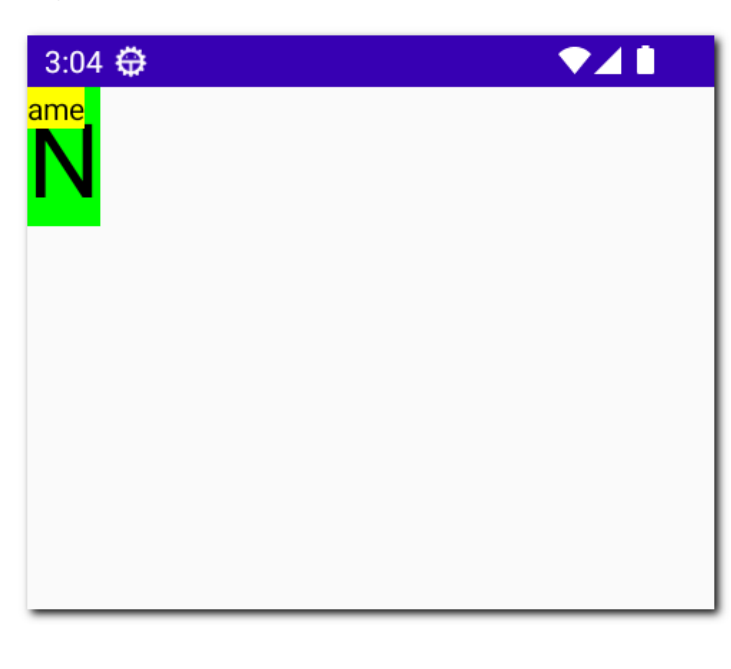
N
ame

Чтобы получить желаемый результат, нам надо, чтобы элементы могли быть наложены друг на друга.
Для этого используем Box вместо Column:

```
import androidx.compose.foundation.layout.Box

@Composable
fun HomeScreen() {
    Box {
        Text(text = "N", fontSize = 48.sp,
            modifier = Modifier.background(color = Color.Green)
        )
        Text(text = "ame",
            modifier = Modifier.background(color = Color.Yellow)
        )
    }
}
```

Результат:



ame
N

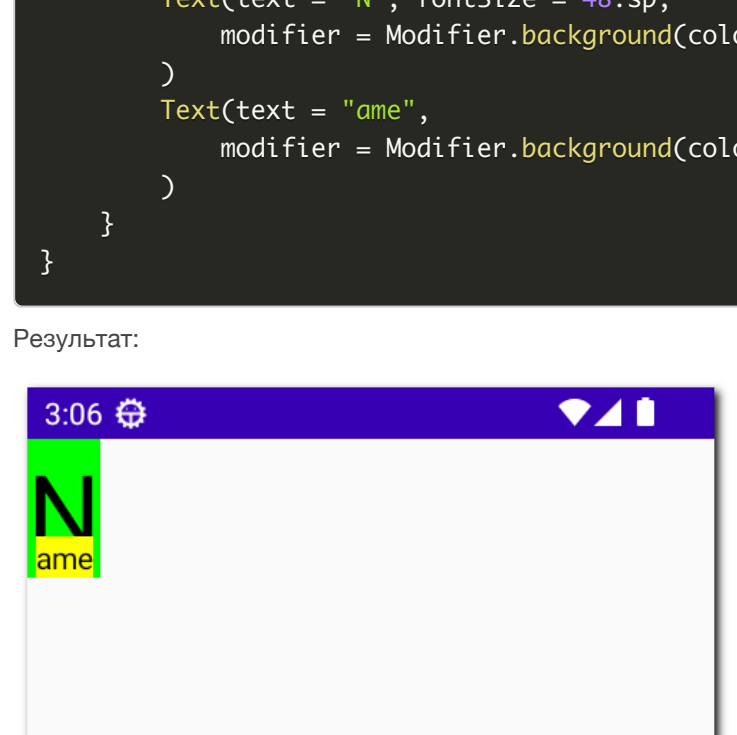
Элементы наложились.

Осталось перетащить вниз и выровнять по центру элемент с текстом "ame". Используем Modifier.align:

```
import androidx.compose.ui.Alignment.Companion.BottomCenter

@Composable
fun HomeScreen() {
    Box {
        Text(text = "N", fontSize = 48.sp,
            modifier = Modifier.background(color = Color.Green)
        )
        Text(text = "ame",
            modifier = Modifier.background(color = Color.Yellow).align(BottomCenter)
        )
    }
}
```

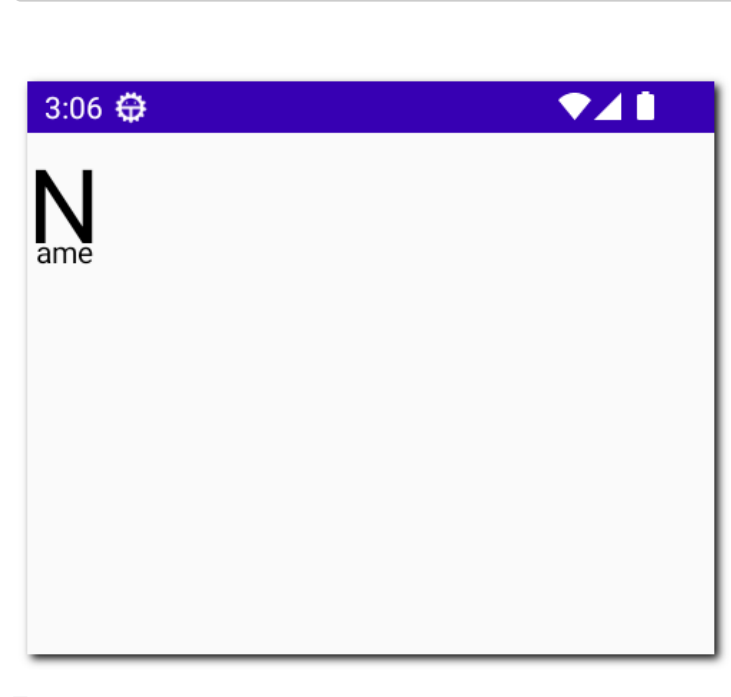
Результат:



N
ame

Убираем фон:

```
@Composable
fun HomeScreen() {
   Box {
       Text(text = "N", fontSize = 48.sp)
       Text(text = "ame",
           modifier = Modifier.align(BottomCenter)
       )
   }
}
```



N
ame

То, что нужно.

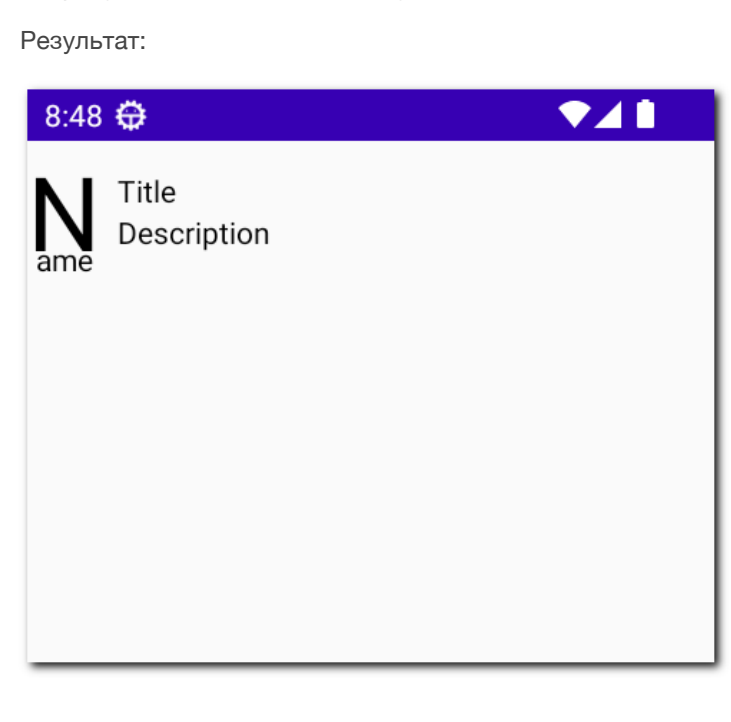
Комбинирование

Чтобы создавать сложный экран, мы комбинируем различные Layout друг с другом:

```
@Composable
fun HomeScreen() {
   Row(verticalAlignment = CenterVertically) {
       Box {
           Text("N", fontSize = 48.sp)
           Text("ame", modifier = Modifier.align(BottomCenter))
       }
       Spacer(modifier = Modifier.width(8.dp))
       Column {
           Text("Title")
           Text("Description")
       }
   }
}
```

В строку мы помещаем Box с двумя текстами и столбец с двумя текстами.

Результат:



N
ame
Title
Description

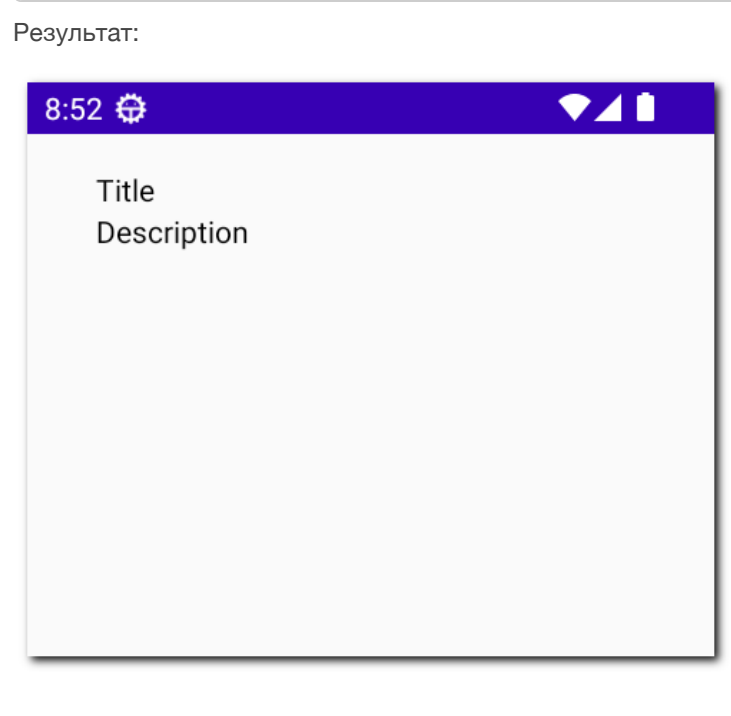
Padding

Чтобы в Layout формировать отступы от границ, используем привычный нам padding. Он добавляется с помощью Modifier.

```
import androidx.compose.foundation.layout.padding
import androidx.compose.ui.unit.dp

@Composable
fun HomeScreen() {
    Column(
        modifier = Modifier.padding(start = 32.dp, top = 16.dp)
    ) {
        Text("Title")
        Text("Description")
    }
}
```

Результат:



Title
Description

Kotlin

Мы используем язык Kotlin, чтобы писать свои Composable функции. А это значит, что мы можем использовать стандартные средства языка, чтобы сформировать экран, который нам нужен. Т.е. мы используем оператор If, чтобы решить, надо ли отображать элемент. Или мы используем цикл, чтобы отобразить несколько одинаковых элементов.

Рассмотрим пример Composable функции:

```
@Composable
fun HomeScreen(list: List<String>) {
   if (list.isEmpty()) {
       Box(contentAlignment = Center, modifier = Modifier.fillMaxSize()) {
           Text(text = "Empty screen")
       }
   } else {
       Column {
           for (s in list) {
               Text(text = s)
           }
       }
   }
}
```

На вход она получает список строк и проверяет пустой ли он. Если список пустой, то HomeScreen выведет текст Empty screen по центру экрана. Если список содержит какие-то данные, то в Column для каждого элемента списка будет вызвана функция Text. В итоге все данные появятся на экране в виде вертикального списка.

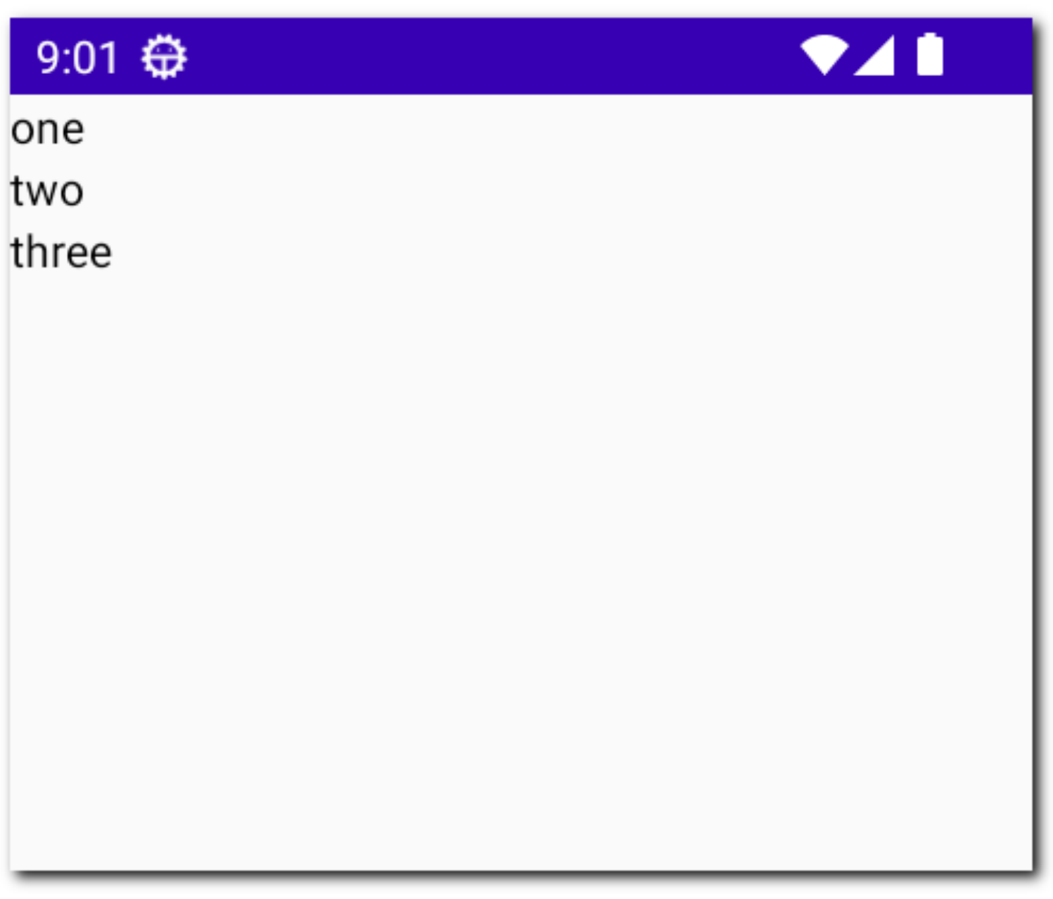
Настоящий список строится по другому, об этом будет отдельный урок.

Проверим, как работает наша функция HomeScreen. Передадим ей простой список строк:

```
class MainActivity : ComponentActivity() {
   override fun onCreate(savedInstanceState: Bundle?) {
       super.onCreate(savedInstanceState)
       setContent {
           HomeScreen(listOf("one", "two", "three"))
```

```
        }
    }
}
```

Результат:

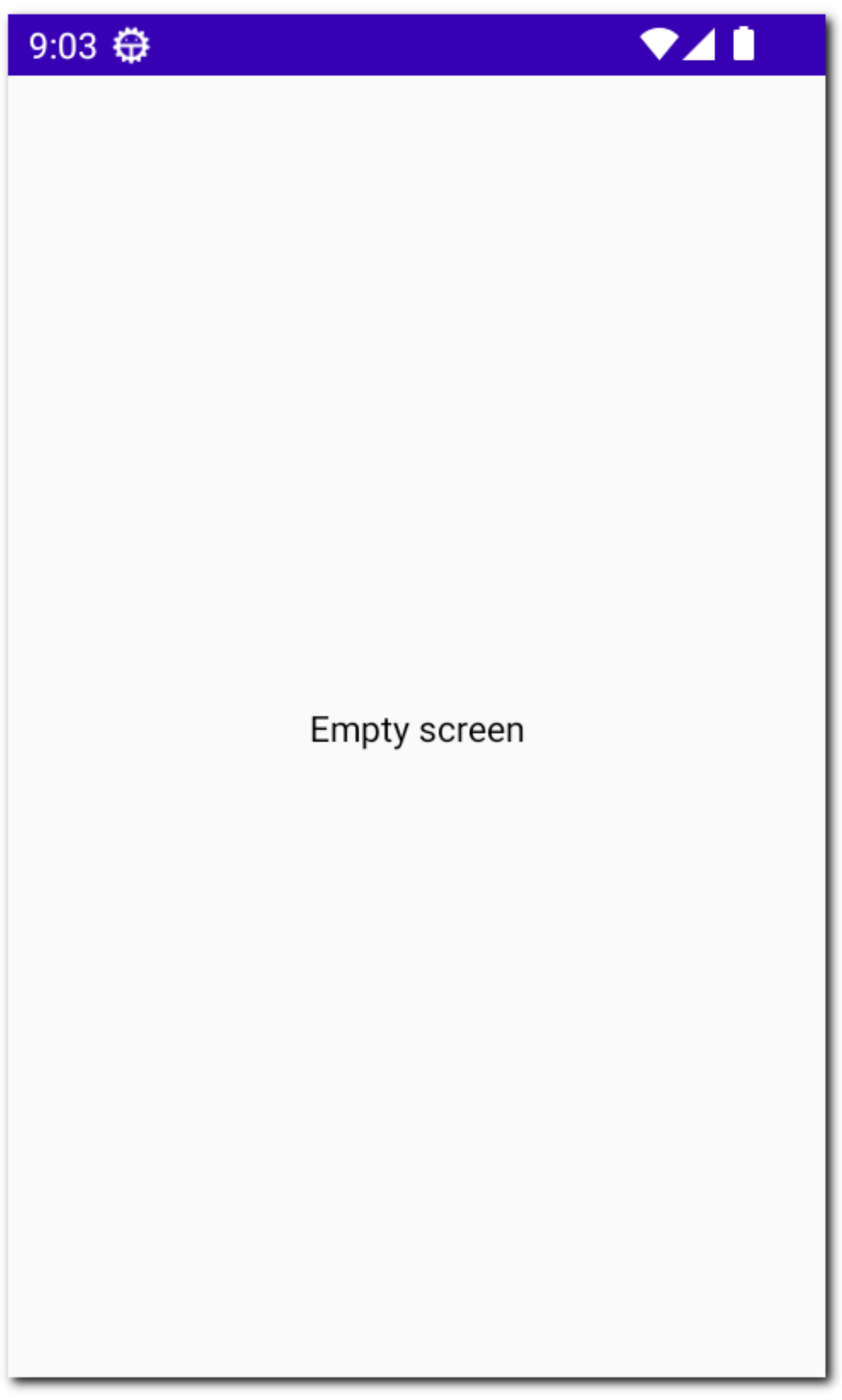

9:01

one
two
three

А если передадим пустой список:

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            HomeScreen(emptyList())
        }
    }
}
```

то увидим пустой экран:


9:03

Empty screen

Таким образом вместо Visibility у нас теперь используется оператор If

Ресурсы

Я для простоты примеров продолжу использовать числа и строки напрямую в коде. Но в реальном приложении мы обычно используем string и dimen ресурсы. Для этого есть функции stringResource и dimensionResource:

```
import androidx.compose.ui.res.dimensionResource
import androidx.compose.ui.res.stringResource

@Composable
fun HomeScreen() {
    Column(
        modifier = Modifier.padding(start = dimensionResource(id = R.dimen.large_padding))
    ) {
        Text(text = stringResource(id = R.string.home_screen_title))
        Text(text = stringResource(id = R.string.home_screen_description))
    }
}
```

Присоединяйтесь к нам в **Telegram**:

- в канале StartAndroid публикуются ссылки на новые статьи с сайта startandroid.ru и интересные материалы с хабра, medium.com и т.п.

- в чатах решаем возникающие вопросы и проблемы по различным темам: Android, Compose, Kotlin, RxJava, Dagger, Тестирование, Performance

- ну и если просто хочется поговорить с коллегами по разработке, то есть чат Флудильня

Назад Вперёд

Комментарии

# **Spacer vs Modifier.padding** — **Татьяна** 06.04.2023 13:57
Добрый день! А с точки зрения производительности, что лучше использовать для отступов, spacer или Modifier.padding?
Ответить

# **RE: Spacer vs Modifier.padding** — **Dmitry Vinogradov** 07.04.2023 16:30
Ну если padding уже есть, то наверно увеличить его будет для системы проще, чем добавлять Spacer. Но скорее всего эта разница на уровне погрешности.
Ответить

# **RE: Spacer vs Modifier.padding** — **Татьяна** 11.04.2023 08:55
Спасибо 🙂
Ответить

Обновить список комментариев
RSS лента комментариев этой записи

Добавить комментарий

Тема

Отправить

JComments

© Start Android, 2011-2020. Все права защищены, копирование материалов возможно только с разрешения автора

При возникновении каких-либо вопросов, используйте форму обратной связи

Bootstrap is a front-end framework of Twitter, Inc. Code licensed under MIT License.
Font Awesome font licensed under SIL OFL 1.1.